Estrutura de Dados

Curso Tecnologia em Análise de Desenvolvimento de Sistemas

Aula 04

Prof. Claudio Benossi

1.
Unidade

Vetores, Matrizes, TAD e Alocação Dinâmica de Memória

Listas e Pilhas (Stacks)

Vamos falar hoje sobre Lista e Pilhas (Stacks).

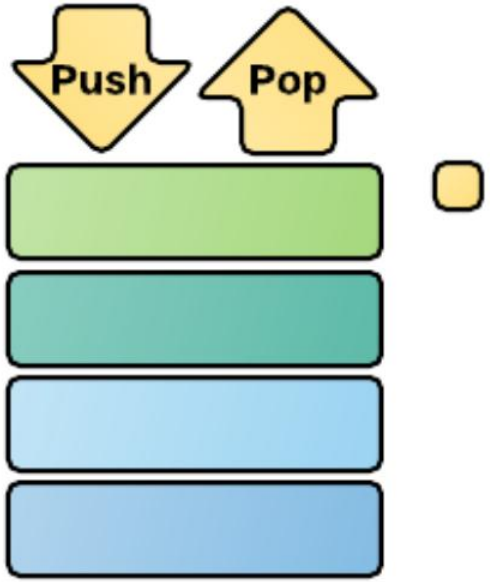
Para muitas aplicações é necessário impor restrições de acesso aos dados armazenados em uma determinada estrutura de dados.

Listas e Pilhas (Stacks)

Essas restrições podem vir a ser benéficas por:

✓ Aliviar a necessidade de usar estruturas com mais detalhes.

✓ Permitem implementações mais simples e flexíveis, já que menos operações são suportadas.



PILHAS - Last In First Out (LIFO).

FILAS - First In First Out (FIFO).

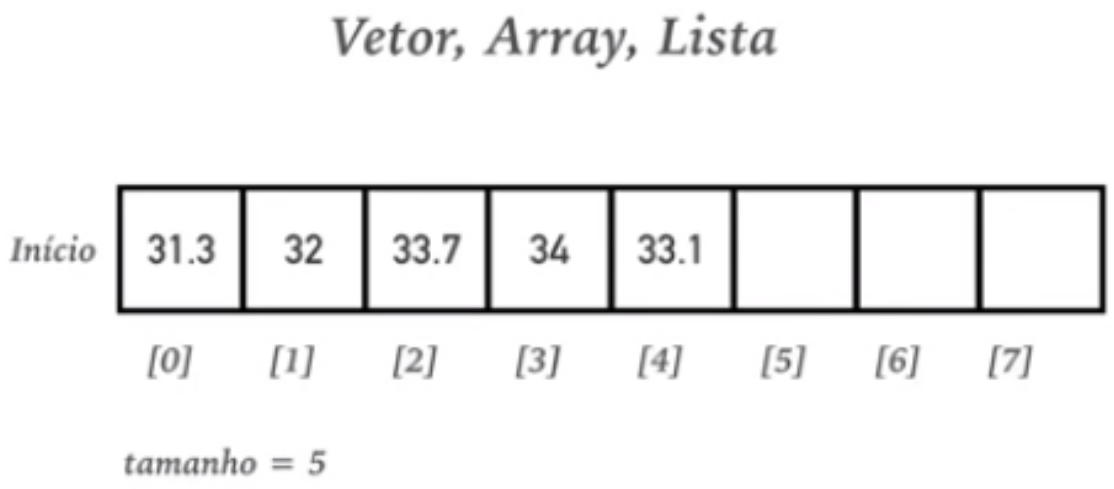
Listas e Pilhas (Stacks)

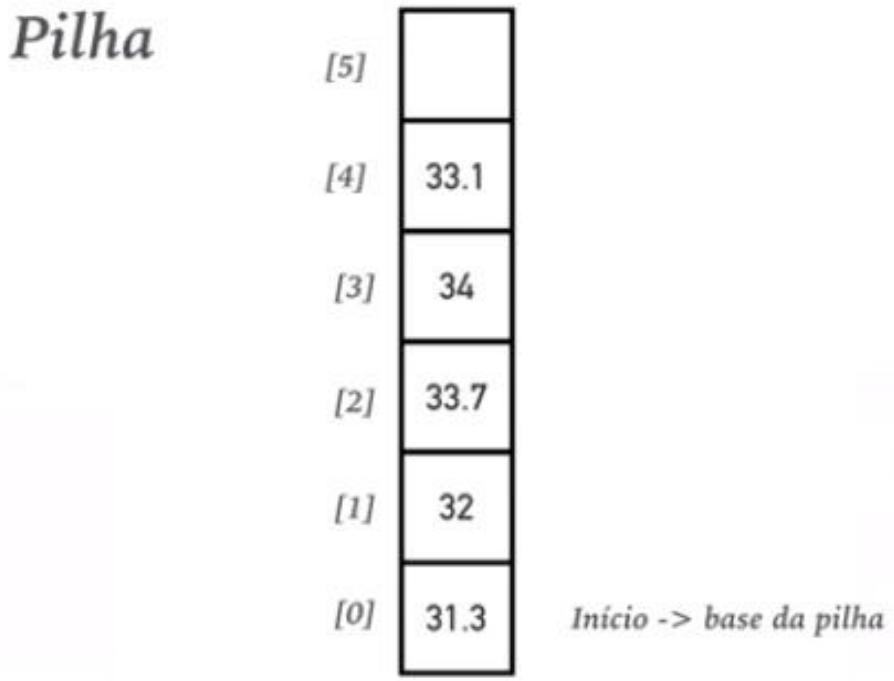
Como uma pilha funciona como estrutura de dados?

Fazendo uma analogia com livros, pratos ou toalhas, podemos escolher qual nós queremos, afinal estão todos no mesmo local, porém um sobre o outro.

Listas e Pilhas (Stacks)

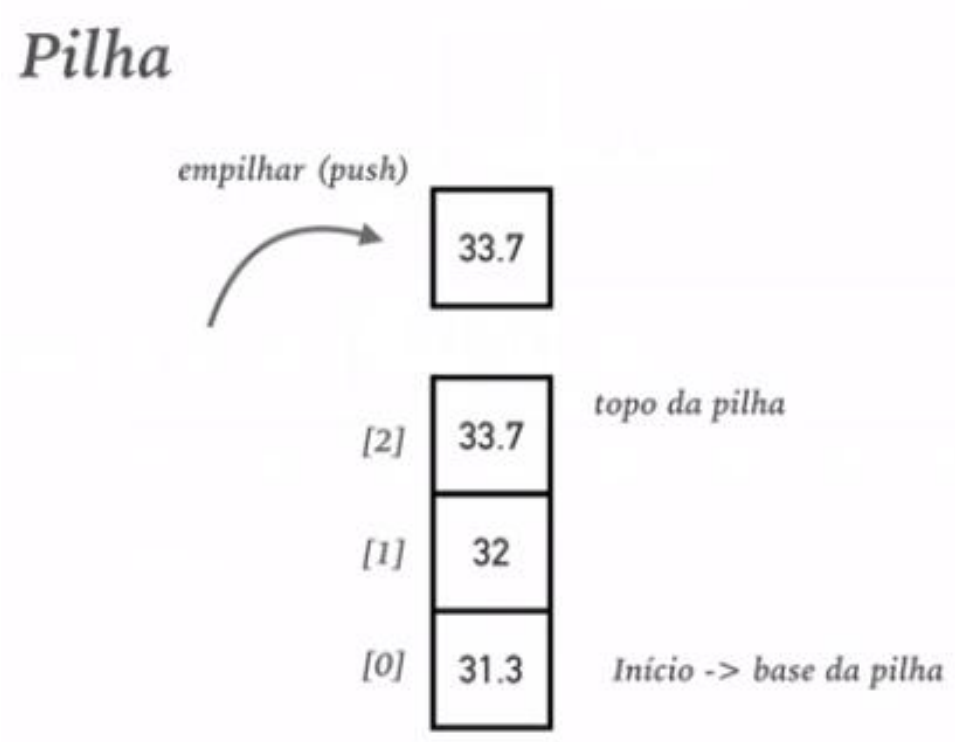
Com relação a estrutura de dados podemos ter:

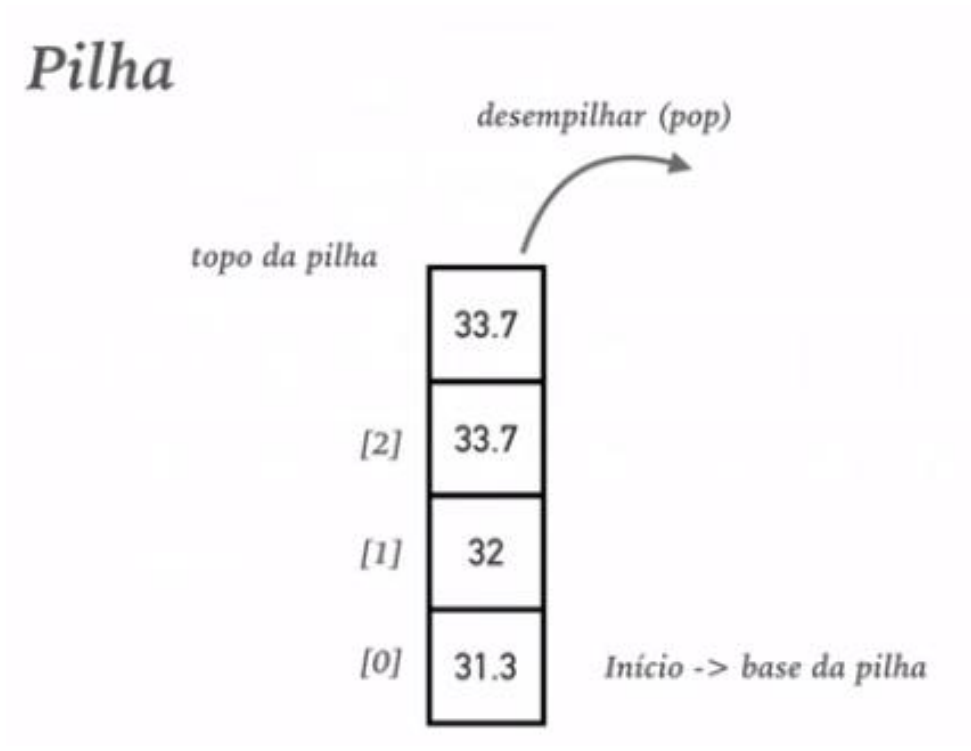




Listas e Pilhas (Stacks)

Como funciona a Pilha:





LIFO – **L**ast **I**n **F**irst **O**ut (Último a entrar, primeiro a sair).

Listas e Pilhas (Stacks)

Com relação ao comportamento da pilha em relação a estrutura de dados temos dois itens importantes que são o push e pop, quando precisamos empilhar adicionamos elementos no topo da pilha, ou seja no final do vetor e quando precisamos desempilhar, nos removemos o ultimo elemento no topo da pilha.

LIFO – **L**ast **I**n **F**irst **O**ut (Último a entrar, primeiro a sair).

Listas e Pilhas (Stacks)

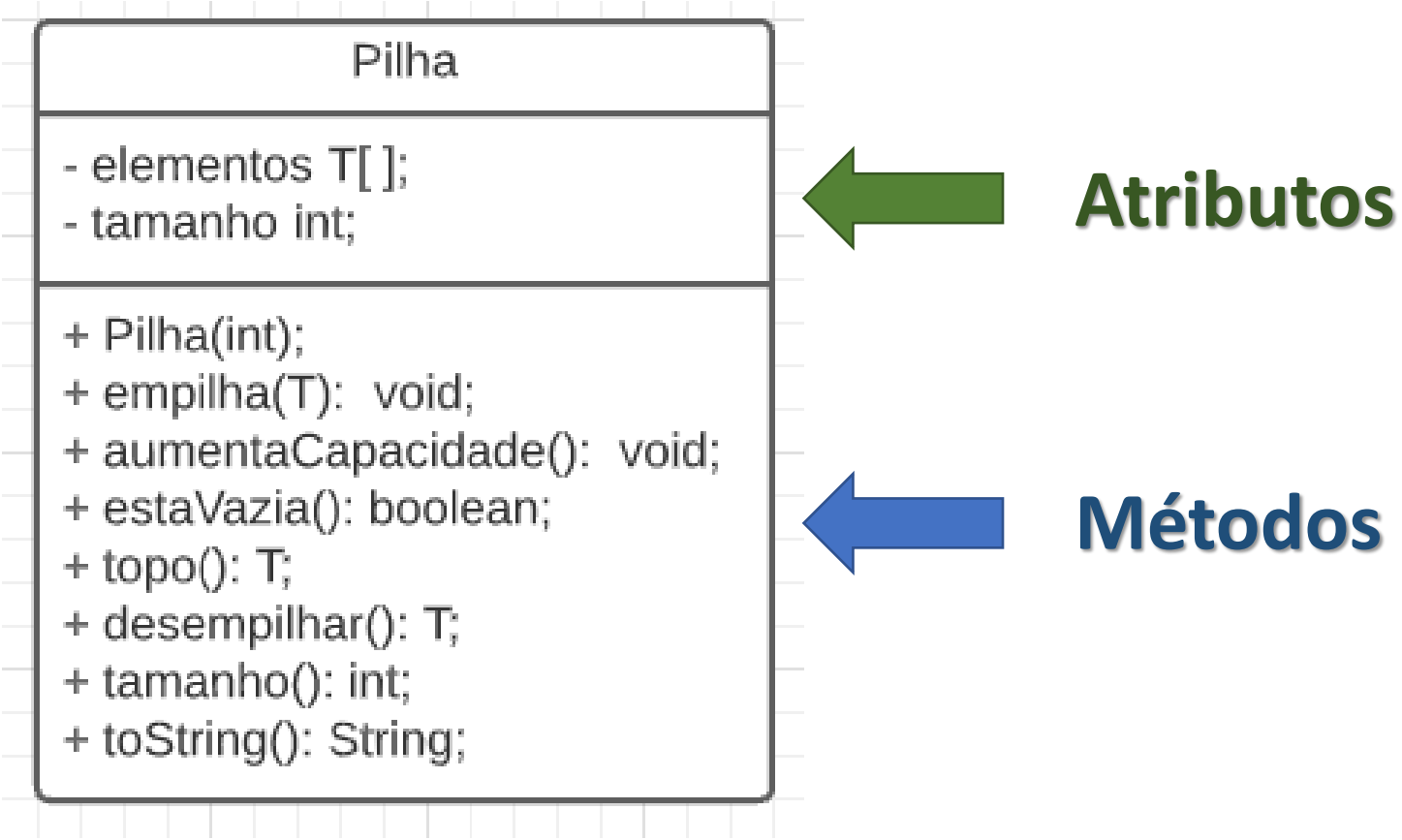
Com relação ao comportamento da pilha em relação a estrutura de dados temos dois itens importantes que são o push e pop, quando precisamos empilhar adicionamos elementos no topo da pilha, ou seja no final do vetor e quando precisamos desempilhar, nos removemos o ultimo elemento no topo da pilha.

LIFO – Last In First Out (Último a entrar, primeiro a sair).

Para entender melhor esse assunto, vamos realizar as seguintes tarefas:

Listas e Pilhas (Stacks)

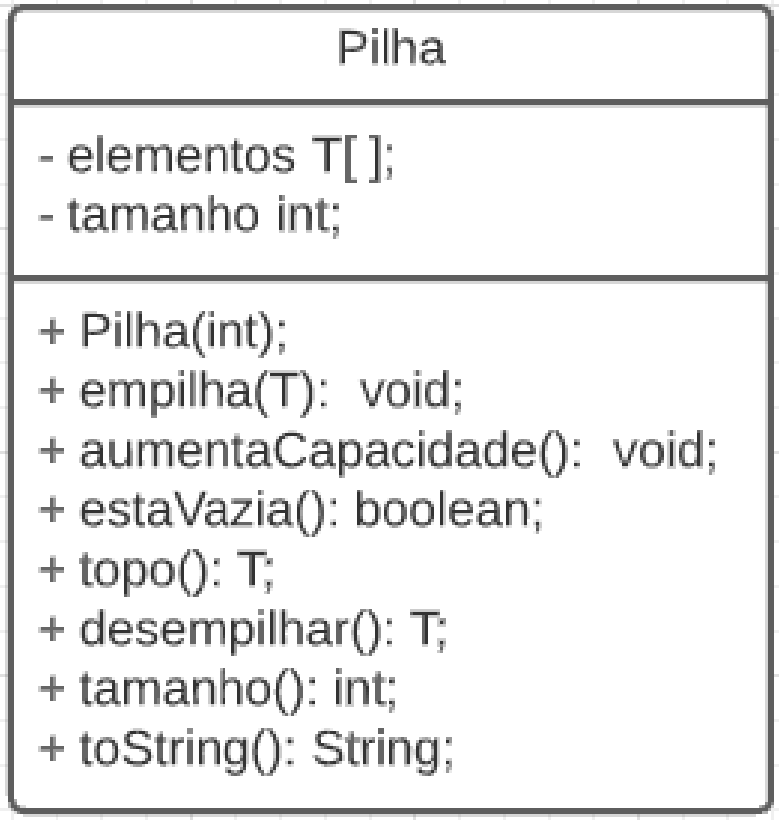
Para exemplificar o conceito de Pilha, vamos seguir a estrutura da classe abaixo:

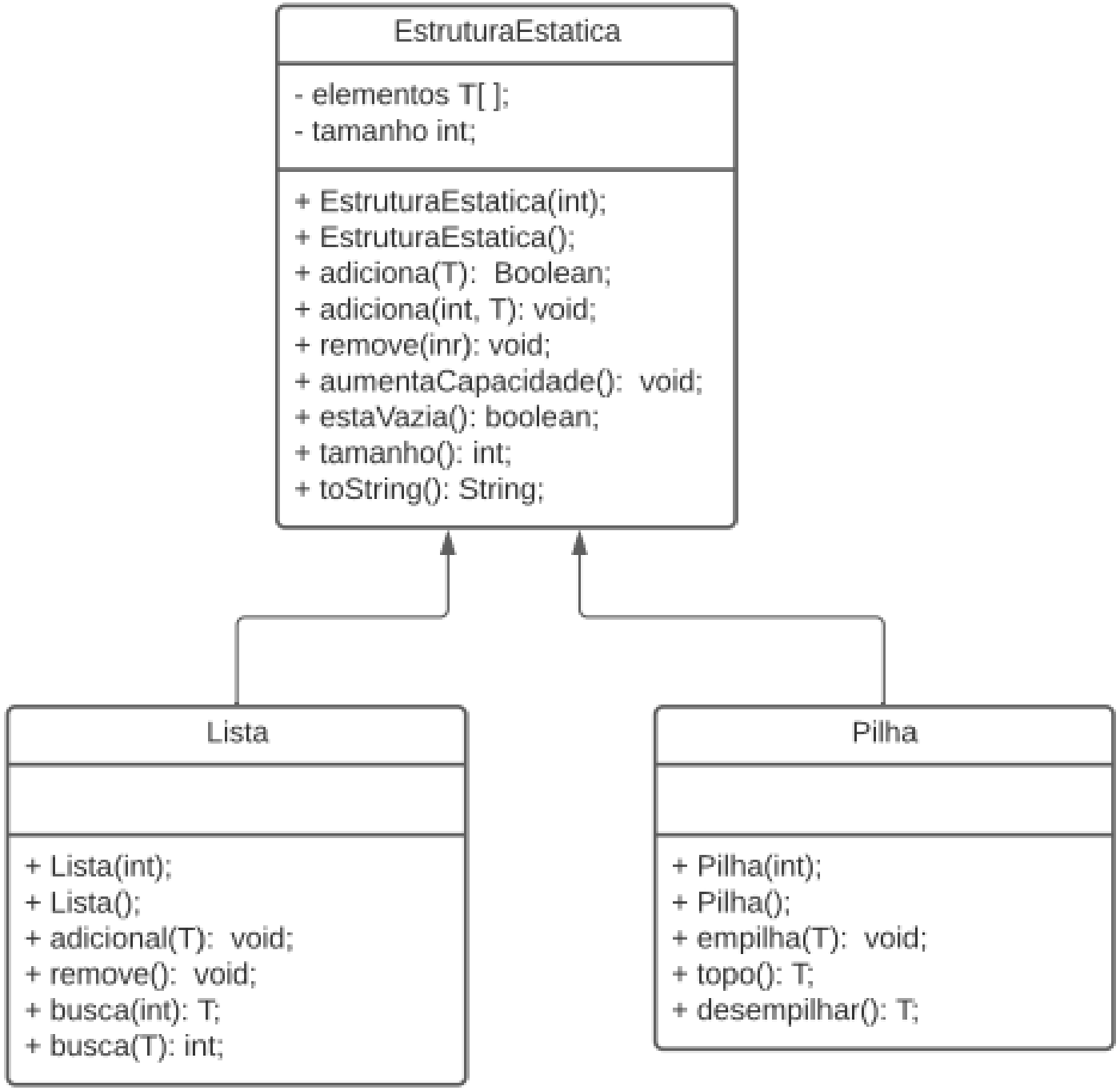


Listas e Pilhas (Stacks)

Obs.: Vamos criar nosso vetor de forma generics “elementos T[ ]”, ou seja, pode ser declarado como qualquer objeto.

Porém para aproveitar nosso projeto e já falar também sobre Listas, vamos trabalhar com um conceito de **herança** já que Pilha e Lista **compartilham** alguns atributos e métodos:




EstruturaEstatica
- elementos T[ ];
- tamanho int;
+ EstruturaEstatica(int);
+ EstruturaEstatica();
+ adiciona(T): Boolean;
+ adiciona(int, T): void;
+ remove(inr): void;
+ aumentaCapacidade(): void;
+ estaVazia(): boolean;
+ tamanho(): int;
+ toString(): String;
Lista
+ Lista(int);
+ Lista();
+ adicional(T): void;
+ remove(): void;
+ busca(int): T;
+ busca(T): int;
Pilha
+ Pilha(int);
+ Pilha();
+ empilha(T): void;
+ topo(): T;
+ desempilhar(): T;

Listas e Pilhas (Stacks)

Vamos começar com a nosso código, primeiro a classe **EstruturaEstatica**, lembrando que vamos reutilizar alguns conceitos vistos na aula anterior sobre vetor.

Main.java × | EstruturaEstatica.java × | Lista.java × | Pilha.java ×

Source | History

```java
package projeto_aula_04_estrutura_estatica;

public class EstruturaEstatica<T> {
    public T[] elementos;
    public int tamanho;

    // Método construtor com parâmetros
    public EstruturaEstatica(int capacidade) {
        this.elementos = (T[]) new Object[capacidade];
        this.tamanho = 0;
    }

    // Método construtor sem parâmetros
    public EstruturaEstatica() {
        this(10);
    }

    // Método para adiconar elementos na estrutura
    public boolean adiciona(T elemento){
        this.aumentaCapacidade();
        this.elementos[this.tamanho] = elemento;
        this.tamanho++;
        return true;
    }
```

Main.java × | EstruturaEstatica.java × | Lista.java × | Pilha.java ×

Source | History

```java
    // Método para adiconar elementos na estrutura em uma posição especifica
    public boolean adiciona(int posicao, T elemento) {
        if (!(posicao >= 0 && posicao < tamanho)) {
            throw new IllegalArgumentException("Posição Inválida");
        }
        this.aumentaCapacidade();
        for (int i = this.tamanho - 1; i >= posicao; i--) {
            this.elementos[i + 1] = this.elementos[i];
        }
        this.elementos[posicao] = elemento;
        this.tamanho++;
        return true;
    }

    // Método para aumentar a capcidade de armazenamento da Estrutura
    public void aumentaCapacidade() {
        if (this.tamanho == this.elementos.length) {
            T[] elementosNovos = (T[]) new Object[this.elementos.length * 2];
            for (int i = 0; i < this.elementos.length; i++) {
                elementosNovos[i] = this.elementos[i];
            }
            this.elementos = elementosNovos;
        }
    }
```

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
    // Método para verificar a quantidade de dados dentro da Estrutura
    public int tamanho() {
        return this.tamanho;
    }

    // Método para exibir os dados da estrutura
    @Override
    public String toString() {
        StringBuilder s = new StringBuilder();
        s.append("[");

        for (int i = 0; i < this.tamanho - 1; i++) {
            s.append(this.elementos[i]);
            s.append(", ");
        }

        if (this.tamanho > 0) {
            s.append(this.elementos[this.tamanho - 1]);
        }

        s.append("]");

        return s.toString();
    }
```

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
77      // Método para verificar se existe elementos na estrutura
78      public boolean estaVazia() {
79          return this.tamanho == 0;
80      }
81
82      // Método para remover elementos da estrutura
83      public void remove(int posicao) {
84          if (!(posicao >= 0 && posicao < tamanho)) {
85              throw new IllegalArgumentException("Posição Inválida");
86          }
87          for (int i = posicao; i < tamanho - 1; i++) {
88              elementos[i] = elementos[i + 1];
89          }
90          tamanho--;
91      }
92
93  }
```

Essa é a Classe Pai ou super Classe.

Agora vamos criar as classes que vão herdar os atributos e os métodos as classes Listas e Pilhas.

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Lista<T> extends EstruturaEstatica<T>{

    // método Construtor com parâmetros o super porque está herdando ...
    public Lista(int capacidade) {
        super(capacidade);
    }

    // método Contrutor sem Parâmetros
    public Lista() {
        super();
    }

    // Método para adiconar elementos na Lista
    public boolean adiciona(T elemento){
        return super.adiciona(elemento);
    }

    // Método para adiconar elementos na Lista em uma posição especifica
    public boolean adiciona(int posicao, T elemento){
        return super.adiciona(posicao, elemento);
    }
}
```

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Pilha<T> extends EstruturaEstatica<T> {

// método Contrutor sem Parâmetros
    public Pilha(){
        super();
    }

    // método Construtor com parâmetros o super porque está herdando ...
    public Pilha(int capacidade){
        super(capacidade);
    }
}
```

OK, sei que ainda falta alguns métodos, porém vamos testar nosso projeto, o que foi feito até agora, para isso vamos criar uma classe teste.

Listas e Pilhas (Stacks)

Classe Teste:

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Main {

    public static void main(String[] args) {
        Pilha<String> pilha = new Pilha<String>();

        System.out.println("Verifica se a Pilha está vazia: " + pilha.estaVazia());

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());
    }

}
```

Listas e Pilhas (Stacks)

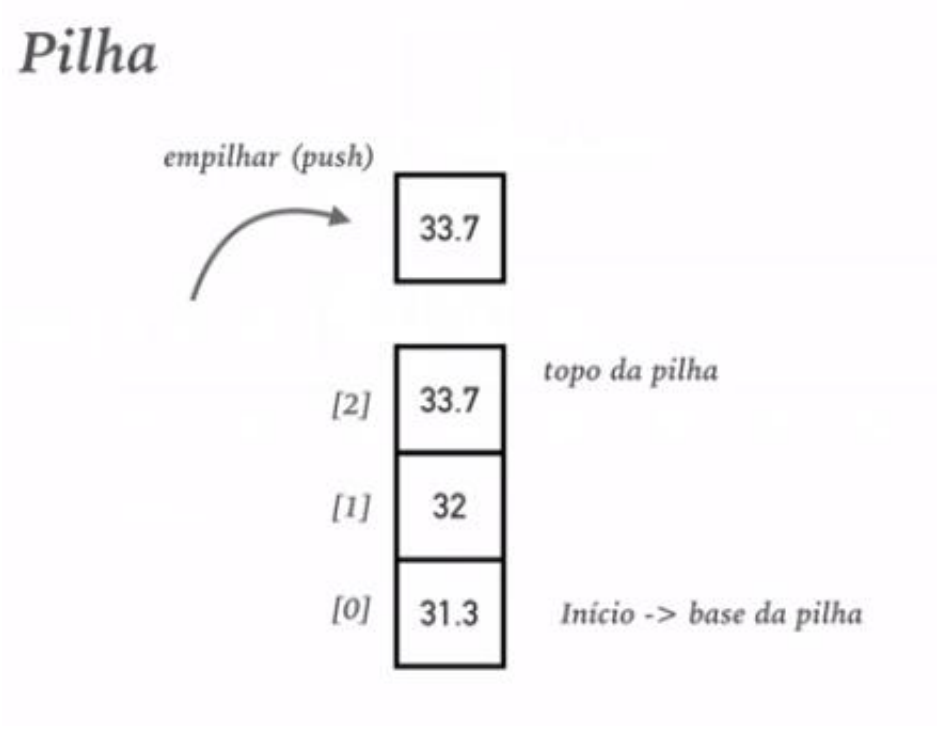
Resultado:

Output - Projeto_Aula_04_Estrutura_Estatica (run)

```
run:
Verifica se a Pilha está vazia: true
Dados da Pilha: []
Qtade de Dados na Pilha: 0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Listas e Pilhas (Stacks)

OK agora que já temos nossa estrutura montada, vamos verificar como empilhar um elemento, lembrando que o comportamento da pilha - **LIFO** – **L**ast **I**n **F**irst **O**ut (Último a entrar, primeiro a sair).



Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
2
3  public class Pilha<T> extends EstruturaEstatica<T> {
4
5  // método Contrutor sem Parâmetros
6      public Pilha(){
7          super();
8      }
9
10     // método Construtor com parâmetros o super porque está herdando ...
11     public Pilha(int capacidade){
12         super(capacidade);
13     }
14
15     // Método para adiconar elementos na Pilha
16     public void empilha(T elemento){
17         // dessa forma estamos reutilizando o código da classe pai
18         super.adiciona(elemento);
19     }
20
21 }
```

Listas e Pilhas (Stacks)

Vamos Testar:

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Main {

    public static void main(String[] args) {
        Pilha<Integer> pilha = new Pilha<Integer>();

        for(int i=1; i<=10; i++){
            pilha.adiciona(i);
        }

        System.out.println("Verifica se a Pilha está vazia: " + pilha.estaVazia());

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());
    }

}
```

Listas e Pilhas (Stacks)

Resultado:

Output - Projeto_Aula_04_Estrutura_Estatica (run)

```
run:
Verifica se a Pilha está vazia: false
Dados da Pilha: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Qtade de Dados na Pilha: 10
BUILD SUCCESSFUL (total time: 0 seconds)
```

Listas e Pilhas (Stacks)

Lembrando que colocamos como parâmetro inicial para criar nosso vetor com 10 posições, porém se dentro da estrutura de repetição for colocarmos:

```
for (int i=1; i<=15; i++){
```

Não teríamos problemas, pois ele verifica o tamanho e aumenta de forma automática.

Listas e Pilhas (Stacks)

Aproveitando para lembrar que a estrutura de dados pilha é muito utilizada pelas próprias linguagens de programação como C#, Java, C, C++, JavaScript, etc., todas elas utilizam uma pilha interna, quando chamamos um método existe uma pilha (stack) de métodos, vale a pena pesquisar sobre o assunto e aproveito para indicar o fórum sobre o assunto mais famoso no mundo que é o ***stackoverflow***, serve para tirar dúvidas, conhecer mais sobre o assunto - https://stackoverflow.com.

Agora vamos verificar se a nossa pilha está vazia:

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Main {

    public static void main(String[] args) {
        Pilha<Integer> pilha = new Pilha<Integer>();

        System.out.println("Verifica se a Pilha está vazia: " + pilha.estaVazia());

        pilha.adiciona(1);

        System.out.println("Verifica se a Pilha está vazia: " + pilha.estaVazia());

    }

}
```

Listas e Pilhas (Stacks)

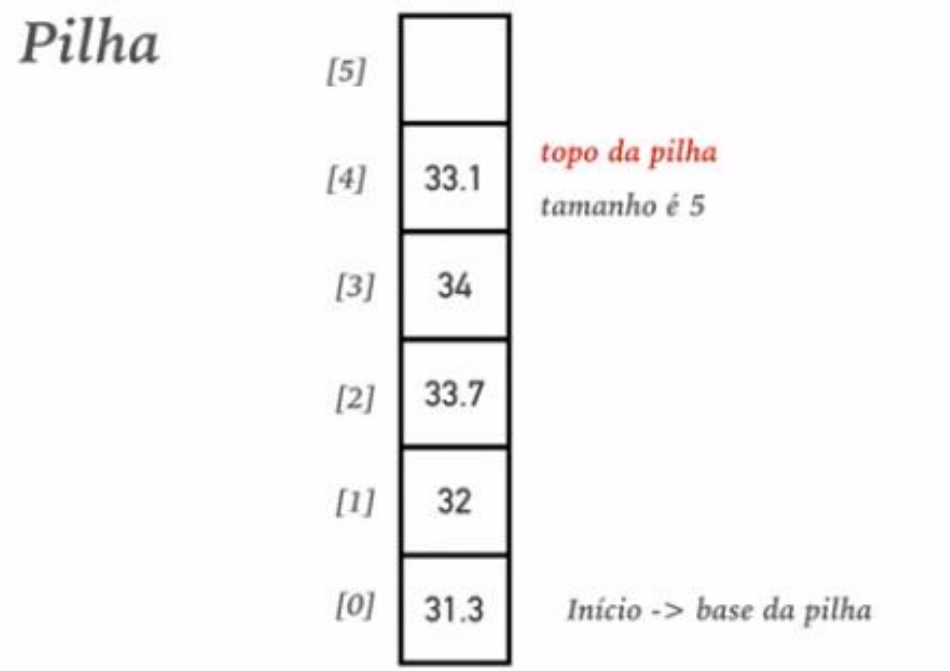
Resultado:

Output - Projeto_Aula_04_Estrutura_Estatica (run)

```
run:
Verifica se a Pilha está vazia: true
Verifica se a Pilha está vazia: false
BUILD SUCCESSFUL (total time: 0 seconds)
```

Listas e Pilhas (Stacks)

Agora vamos verificar o elemento no topo da nossa pilha através do método **topo()**:



Vale lembrar que nesse método eu só vou visualizar a informação, não será inserido e nem removido nenhum elemento.

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
1  package projeto_aula_04_estrutura_estatica;
2
3  public class Pilha<T> extends EstruturaEstatica<T> {
4
5  // método Contrutor sem Parâmetros
6      public Pilha(){...3 lines }
9
10     // método Construtor com parâmetros o super porque está herdando ...
11     public Pilha(int capacidade){...3 lines }
14
15     // Método para adiconar elementos na Pilha
16     public void empilha(T elemento){...4 lines }
20
21     // Método para visualizar a informação no topo da Pilha
22     public T topo(){
23         if(this.estaVazia()){
24             return null;
25         } else {
           return this.elementos[tamanho-1];
27         }
28     }
```

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Main {

    public static void main(String[] args) {
        Pilha<Integer> pilha = new Pilha<Integer>();

        System.out.println("Verifica se a Pilha está vazia: " + pilha.estaVazia());
        System.out.println("Informação no Topo da Pilha: " + pilha.topo());

        pilha.adiciona(1);
        pilha.adiciona(2);
        pilha.adiciona(3);

        System.out.println("Verifica se a Pilha está vazia: " + pilha.estaVazia());
        System.out.println("Informação no Topo da Pilha: " + pilha.topo());

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());
    }
}
```

Listas e Pilhas (Stacks)

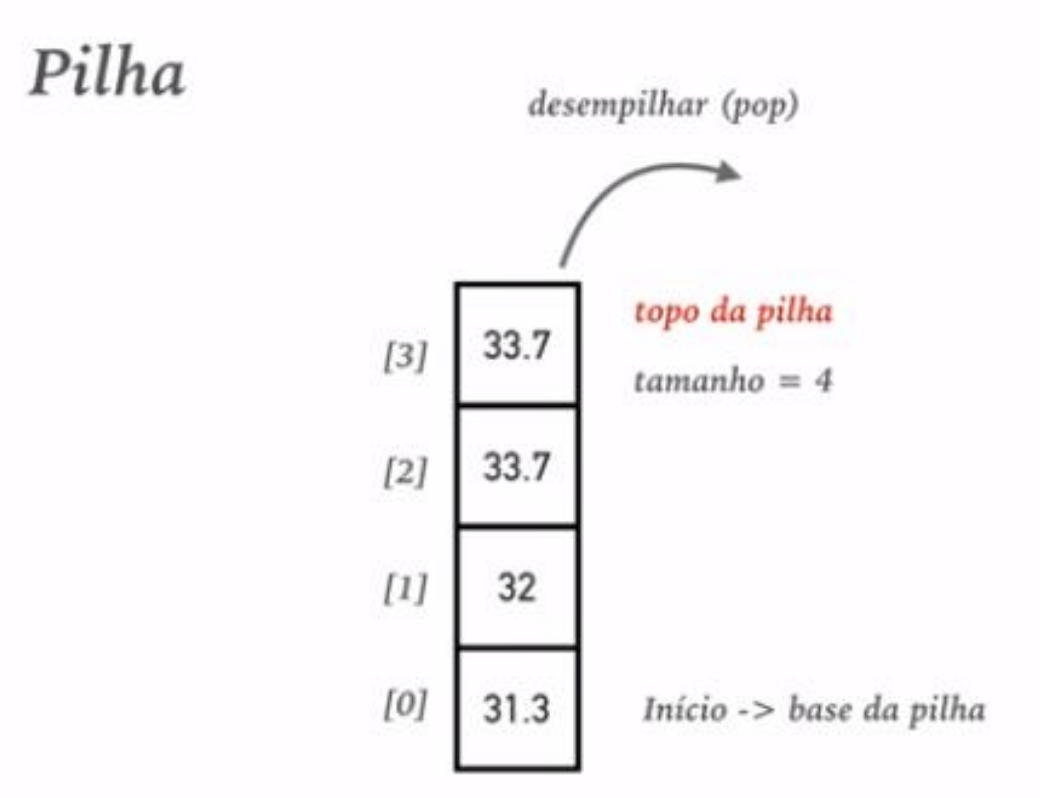
Resultado:

Output - Projeto_Aula_04_Estrutura_Estatica (run)

```
run:
Verifica se a Pilha está vazia: true
Informação no Topo da Pilha: null
Verifica se a Pilha está vazia: false
Informação no Topo da Pilha: 3
Dados da Pilha: [1, 2, 3]
Qtade de Dados na Pilha: 3
BUILD SUCCESSFUL (total time: 0 seconds)
```

Listas e Pilhas (Stacks)

Vamos agora criar o método **desempilhar()** ou seja remover um elemento da nossa pilha, respeitando a característica da pilha **LIFO** – **L**ast **I**n **F**irst **O**ut (Último a entrar, primeiro a sair).



Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```
20
21      // Método para visualizar a informação no topo da Pilha
22      public T topo(){...7 lines }
29
30      // Método para remover um elementos da Pilha
31      public T desempilha(){
32          if(this.estaVazia()){
33              return null;
34          }
35          T elemento = this.elementos[tamanho-1];
36          tamanho--;
37          return elemento;
38      }
39  }
40
```

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Main {

    public static void main(String[] args) {
        Pilha<Integer> pilha = new Pilha<Integer>();

        pilha.adiciona(1);
        pilha.adiciona(2);
        pilha.adiciona(3);

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());

        pilha.desempilha();

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());
    }
}
```

Listas e Pilhas (Stacks)

Agora usando o método remove:

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```
20
21      // Método para visualizar a informação no topo da Pilha
22      public T topo(){...7 lines }
29
30      // Método para remover um elementos da Pilha
31      public T desempilha(){
32          if(this.estaVazia()){
33              return null;
34          }
35          T elemento = this.elementos[tamanho-1];
36          super.remove(tamanho-1);
37          return elemento;
38      }
39  }
```

Main.java × EstruturaEstatica.java × Lista.java × Pilha.java ×

Source History

```java
package projeto_aula_04_estrutura_estatica;

public class Main {

    public static void main(String[] args) {
        Pilha<Integer> pilha = new Pilha<Integer>();

        pilha.adiciona(1);
        pilha.adiciona(2);
        pilha.adiciona(3);

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());

        pilha.desempilha();

        System.out.println("Dados da Pilha: " + pilha.toString());
        System.out.println("Qtade de Dados na Pilha: " + pilha.tamanho());
    }
}
```

Funciona!!!

Talvez seja melhor assim ...

Listas e Pilhas (Stacks)

Resultado:

Output - Projeto_Aula_04_Estrutura_Estatica (run)

```
run:
Dados da Pilha: [1, 2, 3]
Qtade de Dados na Pilha: 3
Dados da Pilha: [1, 2]
Qtade de Dados na Pilha: 2
BUILD SUCCESSFUL (total time: 0 seconds)
```

Listas e Pilhas (Stacks)

Agora que já vimos a lógica do funcionamento de uma pilha, vamos conhecer a **API Java Stack**, que são bem semelhantes aos métodos desenvolvidos até agora nesse projeto.

Teste.java

Código-Fonte | Histórico

```java
package estruturaestatica;

import java.util.Stack;

public class Teste {

    public static void main(String[] args){
        Stack<Integer> stack = new Stack<Integer>();

        // método para verificar se a pilha está vazia
        System.out.println(stack.isEmpty());

        // método para empilhar
        stack.push(1);
        stack.push(2);
        stack.push(3);

        // método para verificar o tamanho da pilha
        System.out.println(stack.size());

        // Método ToString para exibir a pilha
        System.out.println(stack);
```

Teste.java

Código-Fonte | Histórico

```
23
24          // Método para verificar o topo
25          System.out.println(stack.peek());
26
27          // Método para desempilhar
28          System.out.println(stack.pop());
29
30          // Método ToString para exibir a pilha
31          System.out.println(stack);
32
33
34      }
35  }
36
37
```

Listas e Pilhas (Stacks)

Resultado:

Saída

Console do Depurador | EstruturaEstatica (run)

```
run:
true
3
[1, 2, 3]
3
3
[1, 2]
CONSTRUÍDO COM SUCESSO (tempo total: 0 segundos)
```

"Quando a gente acha que tem todas as respostas,
vem a vida e muda todas as perguntas”

Luís Fernando Veríssimo

Obrigado!

Se precisar ...

Prof. Claudio Benossi

claudio.benossi@sp.senac.br